



HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

LẬP TRÌNH C CƠ BẢN



**ĐẠI HỌC
BÁCH KHOA HÀ NỘI**
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

LẬP TRÌNH C CƠ BẢN

CÂY – PHẦN 2

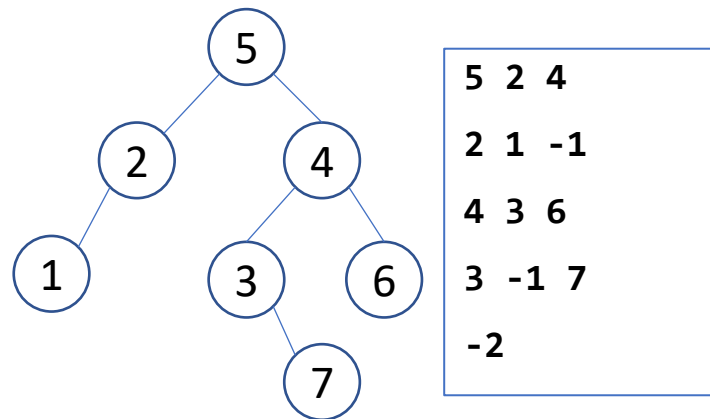
ONE LOVE. ONE FUTURE.

Thao tác với cây nhị phân

- Mỗi nút của cây nhị phân có cấu trúc dữ liệu như sau

```
typedef struct Node{  
    int id; // identifier of the node  
    struct Node* leftChild; // pointer to the left child  
    struct Node* rightChild; // pointer to the right child  
}Node;
```

- Dữ liệu về cây được lưu trong file text với định dạng như sau:
 - Mỗi dòng lưu 3 số nguyên dương t, u, v trong đó u và v (nếu khác -1) sẽ tương ứng là con trái và con phải của t (ghi chú: giá trị t trên mỗi dòng (ngoại trừ dòng 1) là con của 1 phần tử nào đó đã xuất hiện trên dòng trước đó)
 - File kết thúc bởi -2



Thao tác với cây nhị phân

- Lab01. Viết chương trình C chạy trong chế độ tương tác với các lệnh sau:
 - Load <filename>: nạp dữ liệu từ file <filename> và dựng cây
 - Print: In cây ra màn hình
 - AddLeftChild <cur_id> <child_id>: thêm nút con trái <child_id> (nếu chưa tồn tại) vào nút định danh <cur_id> trên cây nếu <cur_id> tồn tại
 - AddRightChild <cur_id> <child_id>: thêm con phải <child_id> (nếu chưa tồn tại) vào nút có định danh <cur_id> vào cây nếu <cur_id> tồn tại
 - Find <id>: tìm nút có định danh <id>
 - Count: Đếm số nút của cây
 - FindLeaves: In ra màn hình các nút lá của cây
 - Height <id>: in ra màn hình chiều cao của nút <id> (nếu tồn tại)
 - Store <filename>: Lưu dữ liệu cây ra file <filename>
 - Quit: thoát khỏi chương trình

Thao tác với cây nhị phân

```
#include <stdio.h>

typedef struct Node{
    int id;
    struct Node* leftChild;
    struct Node* rightChild;
}Node;

Node* root;

Node* makeNode(int id){
    Node* p = (Node*)malloc(sizeof(Node));
    p->id = id;
    p->leftChild = NULL; p->rightChild = NULL;
    return p;
}
```

Thao tác với cây nhị phân

```
Node* find(Node* r, int id){
    if(r == NULL) return NULL;
    if(r->id == id) return r;
    Node* p = find(r->leftChild,id);
    if(p != NULL) return p;
    return find(r->rightChild,id);
}

void addLeftChild(int u, int left){
    Node* pu = find(root,u);
    if(pu == NULL){
        printf("Not found %d\n",u); return;
    }
    if(pu->leftChild != NULL){
        printf("Node %d has already leftChild\n",u); return;
    }
    pu->leftChild = makeNode(left);}
```

Thao tác với cây nhị phân

```
void addRightChild(int u, int right){
    Node* pu = find(root,u);
    if(pu == NULL){
        printf("Not found %d\n",u); return;
    }
    if(pu->rightChild != NULL){
        printf("Node %d has already rightChild\n",u); return;
    }
    pu->rightChild = makeNode(right);
}
```

Thao tác với cây nhị phân

```
void load(char* filename){
    FILE* f = fopen(filename,"r");
    root = NULL;
    while(1){
        int u;
        fscanf(f,"%d",&u);
        if(u == -2) break;// termination
        if(root == NULL) root = makeNode(u);// create the root
        int l,r;
        fscanf(f,"%d%d",&l,&r);
        if(l > -1) addLeftChild(u,l);
        if(r > -1) addRightChild(u,r);
    }
    fclose(f);
}
```

Thao tác với cây nhị phân

```
void printTree(Node* r){
    if(r == NULL) return;
    printf("%d: ",r->id);
    if(r->leftChild == NULL) printf("leftChild = NULL");
    else printf("leftChild = %d",r->leftChild->id);
    if(r->rightChild == NULL) printf(", rightChild = NULL");
    else printf(", rightChild = %d",r->rightChild->id);
    printf("\n");

    printTree(r->leftChild);
    printTree(r->rightChild);
}
```

Thao tác với cây nhị phân

```
void printTreeF(Node* r, FILE* f){
    if(r == NULL) return;
    fprintf(f,"%d ",r->id);
    if(r->leftChild == NULL) fprintf(f,"-1 ");
    else fprintf(f,"%d ",r->leftChild->id);
    if(r->rightChild == NULL) fprintf(f,"-1 ");
    else fprintf(f,"%d ",r->rightChild->id);
    fprintf(f,"\n");

    printTreeF(r->leftChild,f);
    printTreeF(r->rightChild,f);
}
```

Thao tác với cây nhị phân

```
void processLoad(){
    char filename[256];
    scanf("%s",filename);
    load(filename);
}

void printChildren(Node* p){
    if(p->leftChild == NULL) printf(" Node %d does not has leftChild",p->id);
    else printf(", LeftChild = %d",p->leftChild->id);
    if(p->rightChild == NULL) printf(" Node %d does not has rightChild\n",p->id);
    else printf(", RightChild = %d\n",p->rightChild->id);
}
```

Thao tác với cây nhị phân

```
void processFind(){
    int id;
    scanf("%d",&id);
    Node* p = find(root,id);
    if(p == NULL) printf("Not found %d\n",id);
    else {
        printf("Found node %d: ",id);
        printChildren(p);
    }
}

void processPrint(){
    printTree(root);
}
```

Thao tác với cây nhị phân

```
void processAddLeftChild(){
    int id,u;
    scanf("%d%d",&id,&u);
    addLeftChild(id,u);
}
void processAddRightChild(){
    int id,u;
    scanf("%d%d",&id,&u);
    addRightChild(id,u);
}
```

Thao tác với cây nhị phân

```
int height(Node* p){
    if(p == NULL) return 0;
    int maxH = 0;
    int hl = height(p->leftChild);
    if(maxH < hl) maxH = hl;
    int hr = height(p->rightChild);
    if(maxH < hr) maxH = hr;
    return maxH + 1;
}

void processHeight(){
    int id;
    scanf("%d",&id);
    Node* p = find(root,id);
    if(p == NULL) printf("Not found %d\n",id);
    else printf("Height of %d is %d\n",height(p));
}
```

Thao tác với cây nhị phân

```
int count(Node* p){
    if(p == NULL) return 0;
    return 1 + count(p->leftChild) + count(p->rightChild);
}

void printLeaves(Node* p){
    if(p == NULL) return;
    if(p->leftChild == NULL && p->rightChild == NULL)
        printf("%d ", p->id);
    printLeaves(p->leftChild);
    printLeaves(p->rightChild);
}

void processFindLeaves(){
    printLeaves(root); printf("\n");
}
```

Thao tác với cây nhị phân

```
void processCount(){
    printf("Number of nodes = %d\n",count(root));
}

void processStore(){
    char filename[256];
    scanf("%s",filename);
    FILE* f = fopen(filename,"w");
    printTreeF(root,f);
    fprintf(f,"-2");
    fclose(f);
}

void freeTree(Node* r){
    if(r == NULL) return;
    freeTree(r->leftChild);
    freeTree(r->rightChild);
    free(r); r = NULL;
}
```

Thao tác với cây nhị phân

```
void main(){
    while(1){
        char cmd[256]; // representing the input command
        printf("Enter a command: ");
        scanf("%s",cmd);
        if(strcmp(cmd,"Quit") == 0) break;
        else if(strcmp(cmd,"Load")==0) processLoad();
        else if(strcmp(cmd,"Print")==0) processPrint();
        else if(strcmp(cmd,"Find")==0) processFind();
        else if(strcmp(cmd,"Height")==0) processHeight();
        else if(strcmp(cmd,"Count")==0) processCount();
        else if(strcmp(cmd,"FindLeaves")==0) processFindLeaves();
        else if(strcmp(cmd,"AddLeftChild")==0) processAddLeftChild();
        else if(strcmp(cmd,"AddRightChild")==0) processAddRightChild();
        else if(strcmp(cmd,"Store")==0) processStore();
    }
    freeTree(root);
}
```

Thao tác với cây nhị phân

- Yêu cầu thêm
 - Viết hàm tìm và trả về tổ tiên chung gần nhất của 2 nút trên cây
 - Viết hàm in ra các khóa bị trùng nhau trên cây
 - Viết hàm xóa 1 nút trên cây nhị phân
 - Viết hàm giải phóng toàn bộ các nút trên cây nhị phân (bộ nhớ cấp phát động cần được giải phóng)

Xây dựng cây mã huffman

Lab02. Xây dựng cây mã huffman để nén văn bản tiếng Anh

- Cây mã Huffman tạo ra bảng mã hóa dữ liệu sao cho:
 - các ký tự thường xuyên xuất hiện có mã ngắn hơn so với các ký tự ít xuất hiện, giúp giảm kích thước của dữ liệu và tối ưu hóa việc truyền tải, lưu trữ thông tin.
- Quy trình tạo cây mã Huffman:
 - xác định tần suất xuất hiện của từng ký tự hoặc ký hiệu trong dữ liệu.
 - Sau đó, các ký tự này được sắp xếp theo tần suất từ thấp đến cao
 - Bắt đầu với các nút lá tương ứng với các ký tự và tần suất của chúng, quá trình tạo cây tiếp tục bằng cách kết hợp hai nút có tần suất thấp nhất để tạo ra một nút mới.
 - Quá trình này được lặp lại cho đến khi tất cả các nút được kết hợp thành một cây nhị phân hoàn chỉnh.
- Mã Huffman được tạo ra bằng cách duyệt cây từ gốc đến mỗi nút lá, ghi lại đường dẫn từ gốc đến mỗi nút lá để tạo mã nhị phân tương ứng cho từng ký tự.

Xây dựng cây mã huffman

- Cho đầu vào là 1 đoạn văn bản tiếng Anh không dấu chỉ gồm chữ cái thường a-z và dấu cách trống
- Hãy xây dựng cây mã Huffman từ đoạn văn bản đầu vào
- Duyệt cây và in ra chuỗi mã hóa cho từng ký tự
- Tạo văn bản mã hóa từ văn bản ban đầu
- Giải mã văn bản đã mã hóa với cây mã cho trước

Xây dựng cây mã huffman

```
#include <stdio.h>
#include <stdlib.h>
// Định nghĩa một nút trong cây nhị phân
struct TreeNode {
    char data;
    unsigned freq;
    struct TreeNode* left, * right;
};
// Định nghĩa một nút trong danh sách liên kết
// để lưu các cây theo tần số giảm dần
// Nên dùng hàng đợi ưu tiên thay vì danh sách liên kết
struct ListNode {
    struct TreeNode* tNodeData;
    struct ListNode* next;
};
```

```
// Tạo một nút mới cho cây Huffman
struct TreeNode* newNode(char data, unsigned freq) {
    struct TreeNode* temp = (struct
    TreeNode*)malloc(sizeof(struct TreeNode));
    temp->left = temp->right = NULL;
    temp->data = data;
    temp->freq = freq;
    return temp;
}
```

Xây dựng cây mã huffman

```
// Hàm thêm nút vào danh sách liên kết đơn theo tần suất tăng dần nút mới thêm là node và đầu danh sách hiện tại là head
struct ListNode* insertNode(struct ListNode* head, struct TreeNode* node){
    struct ListNode* newNode = (struct ListNode*)malloc(sizeof(struct ListNode));
    newNode->tNodeData = node;
    newNode->next = NULL;
    if (head == NULL) { // nếu danh sách hiện tại rỗng thì nút là đầu
        return newNode;
    }
    // tìm vị trí thêm // TH1. thêm vào trước tiên
    if (head->tNodeData->freq > newNode->tNodeData->freq) {
        newNode->next = head;
        head = newNode;
    }
    else { // TH2 tìm vị trí thêm ở sau
        struct ListNode* preNode = head;
        while (preNode->next != NULL && preNode->next->tNodeData->freq < newNode->tNodeData->freq) {
            preNode = preNode->next;
        }
        newNode->next = preNode->next;
        preNode->next = newNode;
    }
    return head;
}
```

Xây dựng cây mã huffman

```
// hàm lấy nút ở đầu danh sách
// danh sách chắc chắn có phần tử nên không cần check
struct TreeNode* getTreeNodeFromList(struct ListNode** head)
{
    struct ListNode* temp = *head;
    *head = (*head)->next;

    struct TreeNode* p = temp->tNodeData;
    free(temp);

    return p;
}
```

Xây dựng cây mã huffman

```
// Hàm xây dựng cây Huffman từ danh sách tần suất
struct TreeNode* buildHuffmanTree(char data[], int freq[], int size) {
    struct ListNode* listHead = NULL;
    struct TreeNode* tNode = NULL, * left, * right;

    // thêm các ký tự vào danh sách
    for (int i = 0; i < size; ++i) {
        tNode = newNode(data[i], freq[i]);
        listHead = insertNode(listHead, tNode);
    }
    while (listHead != NULL && listHead->next != NULL) {
        left = getTreeNodeFromList(&listHead);
        right = getTreeNodeFromList(&listHead);

        // Nút tổng sẽ đại diện là ký tự $
        tNode = newNode('$', left->freq + right->freq);
        tNode->left = left;
        tNode->right = right;
        listHead = insertNode(listHead, tNode);
    }
    return getTreeNodeFromList(&listHead);
}
```

Xây dựng cây mã huffman

```
// Hàm in encode với trái là 0 và phải là bit 1
void printKeyEncode(struct TreeNode* root, char word[], int level) {
    if (root == NULL) return;
    if (root->left==NULL && root->right==NULL) {
        word[level] = '\0';
        printf("%c : %s\n", root->data, word);
        return;
    }
    if (root->left != NULL)
    {
        word[level] = '0';
        printKeyEncode(root->left, word, level + 1);
    }
    if (root->right != NULL)
    {
        word[level] = '1';
        printKeyEncode(root->right, word, level + 1);
    }
}
```

Xây dựng cây mã huffman

```
// In thứ tự tiền thứ tự của cây Huffman
void printPreOrder(struct TreeNode* root) {
    if (root != NULL) {
        printf("(%c, %u) ", root->data, root->freq);
        printPreOrder(root->left);
        printPreOrder(root->right);
    }
}

// Hàm giải phóng bộ nhớ của cây
void freeTree(struct TreeNode* root) {
    if (root != NULL) {
        freeTree(root->left);
        freeTree(root->right);
        free(root);
    }
}
```

Xây dựng cây mã huffman

```
int main() {
    char arr[] = { 'a', 'b', 'c', 'd', 'e', 'f' };
    int freq[] = { 5, 9, 12, 13, 16, 45 };
    int size = sizeof(arr) / sizeof(arr[0]);
    char word[50];
    int level = 0;
    struct TreeNode* root = buildHuffmanTree(arr, freq, size);

    printf("Pre-order traversal of the Huffman tree:\n");
    printPreOrder(root);
    printf("\n");

    printf("Key encode\n");
    printKeyEncode(root, word, level);

    freeTree(root);

    return 0;
}
```

A large graphic on the left side of the slide. It features a dark blue background with a circular pattern of red dots of varying sizes, creating a sense of depth and movement. The word "HUST" is centered within this graphic in a bold, white, sans-serif font.

HUST

THANK YOU !